Universidade Estadual de Santa Cruz – UESC

Relatórios de Implementações de Métodos da Disciplina Análise Numérica

Relatório de implementações realizadas por Ariel Pina Ribeiro

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2025.2

Professor Gesil Sampaio Amarante II

Ilhéus – BA
2025

ÍNDICE

ÍNDICE **2**
Linguagem(ns) Escolhida(s) e justificativas **4**
Método de Euler **5**
Estratégia de Implementação: 5
Estrutura dos Arquivos de Entrada/Saída 5
Dificuldades enfrentadas 6
Método de Euler Modificado **7**
Estratégia de Implementação: 7
Estrutura dos Arquivos de Entrada/Saída 7
Dificuldades enfrentadas 8
Método de Heun **9**
Estratégia de Implementação: 9
Estrutura dos Arquivos de Entrada/Saída 9
Dificuldades enfrentadas 10
Método de Ralston **12**
Estratégia de Implementação: 12
Estrutura dos Arquivos de Entrada/Saída 13
Dificuldades enfrentadas 13
Método de Runge-Kutta de 3° ordem **15**
Estratégia de Implementação: 15
Estrutura dos Arquivos de Entrada/Saída 16
Dificuldades enfrentadas 16
Método de Runge-Kutta de 4° ordem **18**
Estratégia de Implementação: 18
Estrutura dos Arquivos de Entrada/Saída 19
Dificuldades enfrentadas 19
Problemas Teste (Propostos pelo professor) **21**
Problema 12.2: 21
Entrada: 21
Saídas 21
Saída por Euler: 22
Saída por Euler Modificado: 22
Saída por Heun: 23
Saída por Ralston: 24
Saída por Runge-Kutta de 3° ordem: 24
Saída por Runge-Kutta de 4° ordem: 25
Problema 12.12: 26
Entrada: 26

Saídas..... 26
Saída por Euler:..... 26
Saída por Euler Modificado:..... 27
Saída por Heun:..... 28
Saída por Ralston:..... 28
Saída por Runge-Kutta de 3° ordem:..... 29
Saída por Runge-Kutta de 4° ordem:..... 30
Problema 12.16:..... 31
Entrada:..... 31
Saídas..... 31
Saída por Euler:..... 32
Saída por Euler Modificado:..... 32
Saída por Heun:..... 33
Saída por Ralston:..... 34
Saída por Runge-Kutta de 3° ordem:..... 34
Saída por Runge-Kutta de 4° ordem:..... 35
Método Shooting..... 36
Estratégia de Implementação:..... 36
Estrutura dos Arquivos de Entrada/Saída..... 37
Dificuldades enfrentadas..... 37
Método de Diferenças Finitas..... 39
Estratégia de Implementação:..... 39
Estrutura dos Arquivos de Entrada/Saída..... 40
Dificuldades enfrentadas..... 40
Problemas Teste (Propostos pelo professor)..... 42
Problema 1 (Método “Shooting”)..... 42
Entrada:..... 42
Saída:..... 43
Problema 2 (Método de Diferenças Finitas)..... 44
Entrada:..... 44
Saída:..... 44

Linguagem(ns) Escolhida(s) e justificativas

A linguagem escolhida para as implementações deste relatório foi a linguagem Python. A opção por essa linguagem foi por conta de sua sintaxe simples, que facilita a transformação de fórmulas matemáticas em código e o entendimento dos algoritmos. Além disso, a linguagem possui uma ampla gama de bibliotecas integradas, que facilitam as implementações dos algoritmos.

Entre essas bibliotecas, podemos utilizar NumPy para operações com vetores e matrizes, SciPy para funções numéricas prontas, SymPy quando precisarmos de manipulação simbólica e Matplotlib para gráficos. Algumas dessas bibliotecas podem precisar ser instaladas no computador ou ambiente virtual Python que estiver rodando o código.

Outro ponto importante é que Python facilita a leitura e escrita de arquivos de entrada e saída em formatos simples (texto, CSV) e a geração automática de relatórios e gráficos.

Além de ser a linguagem recomendada pelo professor, as características citadas acima tornam a linguagem adequada para implementar, testar e comparar os métodos estudados, permitindo também, se necessário, otimizações futuras sem alterar a lógica matemática das soluções.

Método de Euler

Estratégia de Implementação:

O método de Euler foi implementado seguindo seu padrão clássico para resolver equações diferenciais ordinárias de primeira ordem. A abordagem consiste em utilizar a derivada da função em um ponto para estimar seu valor no ponto seguinte, avançando em passos de tamanho fixo ao longo do intervalo de integração.

A implementação foi estruturada em módulos distintos para garantir a reutilização de código. A função principal do método recebe a expressão da derivada (em formato de string), os valores iniciais de x e y, o tamanho do passo h e o número de iterações desejadas. A cada iteração, o valor de y é atualizado pela fórmula $yi + 1 = yi + h * f(xi, yi)$, enquanto x avança pelo passo h.

Para interpretar a expressão da derivada fornecida pelo usuário, foi utilizado um avaliador de expressões que reconhece operadores matemáticos básicos e funções da biblioteca math do Python. Isso permite que o usuário insira funções como "x + y", "math.sin(x) - y" ou "2*x**2 + y/3" de forma flexível, sem necessidade de recompilar o código.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** segue um formato simples e direto, contendo em cada linha as informações necessárias para a execução do método:

1. Expressão da função diferencial f(x, y) em formato compatível com Python
2. Valor inicial de x (x_0)
3. Valor inicial de y (y_0)
4. Tamanho do passo de integração (h)
5. Número de iterações (n)

O arquivo de saída (**saida.txt**) é gerado automaticamente após a execução do método e contém:

1. Uma linha inicial indicando o método utilizado: "Método: Euler"
2. Sequência de linhas com os pares (x, y) calculados em cada iteração, formatados com seis casas decimais

Dificuldades enfrentadas

A principal dificuldade foi garantir a interpretação correta da expressão da derivada fornecida como string, exigindo um tratamento de possíveis erros de sintaxe e validação dos operadores matemáticos permitidos. Outro desafio foi a escolha adequada do tamanho do passo h, pois valores muito grandes podem levar a imprecisões acumuladas, enquanto valores muito pequenos aumentam o tempo de processamento sem ganhos perceptíveis de precisão.

Além disso, a necessidade de entrada formatada de maneira rigorosa (uma informação por linha, na ordem exata) fez com que fosse necessária a implementação de verificações de consistência dos dados lidos do arquivo. Por fim, a observação do erro de truncamento acumulado, reforçou a importância de considerar métodos de ordem superior para problemas que exigem maior exatidão.

Método de Euler Modificado

Estratégia de Implementação:

O Método de Euler Modificado foi implementado como uma melhoria do método de Euler clássico, visando aumentar a precisão das aproximações para equações diferenciais ordinárias de primeira ordem. A abordagem escolhida utiliza uma técnica de previsão e correção, onde são calculadas duas estimativas da inclinação da solução em cada passo do código.

A implementação segue a fórmula de atualização baseada em uma média ponderada das inclinações. Primeiramente, é calculada a inclinação inicial $k1 = f(xi, yi)$ no ponto atual. Em seguida, uma previsão do valor da função no próximo ponto é obtida, e essa previsão é utilizada para calcular uma segunda inclinação k2. O valor final de yi+1 é, então, determinado pela média dessas duas inclinações.

A estrutura do código foi organizada de forma modular, aproveitando as mesmas funções de leitura de entrada e gravação de saída utilizadas no método de Euler clássico, garantindo uma implementação mais fácil e consistente. O núcleo do método está encapsulado em uma função específica que recebe a função derivada, condições iniciais, passo e número de iterações, retornando as listas de valores de x e y aproximados.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** segue um formato simples e direto, contendo em cada linha as informações necessárias para a execução do método:

1. Expressão da função diferencial f(x, y) em formato compatível com Python
2. Valor inicial de x (x_0)
3. Valor inicial de y (y_0)
4. Tamanho do passo de integração (h)
5. Número de iterações (n)

O arquivo de saída (**saida.txt**) é gerado automaticamente após a execução do método e contém:

1. Uma linha inicial indicando o método utilizado: "Método: Euler modificado"
2. Sequência de linhas com os pares (x, y) calculados em cada iteração, formatados com seis casas decimais

Dificuldades enfrentadas

A principal dificuldade encontrada foi garantir que a implementação da lógica de previsão e correção estivesse correta, assegurando que a segunda inclinação k2 fosse calculada utilizando uma previsão adequada do próximo ponto. O entendimento da diferença entre o método de Euler clássico e sua versão modificada exigiu uma atenção especial, pois embora ambos sejam de primeira ordem, o Euler Modificado apresenta menor erro de truncamento.

Outro aspecto foi a validação dos resultados, comparando-os com soluções analíticas já conhecidas para verificar se a melhoria de precisão era realmente alcançada. Em alguns casos, foi necessário ajustar o tamanho do passo para observar diferenças significativas entre os métodos. A estrutura modular do código facilitou a comparação direta entre as implementações, permitindo verificar a consistência dos resultados obtidos.

Além disso, a necessidade de interpretar expressões matemáticas fornecidas como string demandou cuidados especiais com o escopo de variáveis e funções disponíveis durante a avaliação, garantindo que apenas operações seguras e permitidas pudessem ser executadas.

Método de Heun

Estratégia de Implementação:

O método de Heun foi implementado como uma melhoria do método de Euler convencional, visando aumentar a precisão na solução de equações diferenciais ordinárias. Ao contrário do método de Euler clássico, que utiliza apenas a inclinação no início do intervalo, o método de Heun usa uma abordagem de previsão e correção que considera também a inclinação no ponto final estimado.

A lógica central do algoritmo se baseia no cálculo de duas estimativas de inclinação:

- k_1: Representa a inclinação no ponto atual (x_n, y_n)
- k_2: Representa a inclinação no ponto previsto (x_{n+1}, $y_n + h \cdot k_1$)

Após obter essas duas inclinações, o valor de y é atualizado utilizando uma média ponderada entre k_1 e k_2, especificamente através da fórmula: $y_{n+1} = y_n + (h/2) \cdot (k_1 + k_2)$.

Na implementação prática, implementei o código para manter a mesma interface dos demais métodos numéricos. A função heun() recebe como parâmetros a função diferencial f(x, y), os valores iniciais x0 e y0, o tamanho do passo h e o número de iterações n. Durante a execução, os valores de x e y são armazenados em listas separadas que são atualizadas a cada iteração, mantendo o histórico completo da solução aproximada.

Uma decisão importante foi reutilizar a infraestrutura já desenvolvida para os outros métodos (embora em códigos diferentes), como as funções de leitura do arquivo de entrada e gravação do arquivo de saída. Isso garantiu consistência e facilitou a comparação entre diferentes métodos numéricos. A função que implementa o método de Heun ficou modular e independente, podendo ser chamada de forma isolada ou integrada ao sistema principal.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** segue um formato simples e direto, contendo em cada linha as informações necessárias para a execução do método:

1. Expressão da função diferencial f(x, y) em formato compatível com Python
2. Valor inicial de x (x_0)
3. Valor inicial de y (y_0)
4. Tamanho do passo de integração (h)
5. Número de iterações (n)

O arquivo de saída (**saida.txt**) é gerado automaticamente após a execução do método e contém:

1. Uma linha inicial indicando o método utilizado: "Método: Heun"
2. Sequência de linhas com os pares (x, y) calculados em cada iteração, formatados com seis casas decimais

Dificuldades enfrentadas

A implementação do método de Heun apresentou alguns desafios interessantes durante o desenvolvimento. Embora a lógica do método seja relativamente direta como uma evolução do método de Euler, para garantir sua correta implementação foi necessário uma certa atenção a alguns detalhes específicos. Uma das principais preocupações foi garantir que as duas inclinações, k_1 e k_2, fossem calculadas corretamente, especialmente porque o valor de k_2 depende de uma estimativa inicial baseada em k_1. Essa dependência sequencial significava que qualquer pequeno erro no cálculo inicial poderia comprometer significativamente os resultados finais posteriores.

A validação dos resultados também demandou tempo e uma certa atenção. Pois era necessário ter certeza de que a implementação estava correta, logo, precisei comparar os resultados com soluções conhecidas e com as saídas de outros métodos. Em alguns testes iniciais, pequenas discrepâncias apareceram, que depois de uma análise, descobri que estavam relacionadas a questões de arredondamento e precisão numérica. Ajustar o código para lidar adequadamente com essas nuances foi essencial para garantir a confiabilidade do método.

Por fim, uma dificuldade sutil mas importante foi garantir que o método de Heun fosse claramente distinguido de outras abordagens similares, como o método de Euler Modificado. As fórmulas podem parecer semelhantes à primeira vista, mas suas fundamentações teóricas e comportamentos práticos têm diferenças importantes. Revisar a teoria com cuidado e implementar cada método com

suas características específicas foi necessário para evitar confusões e garantir que cada algoritmo mantivesse sua identidade e propriedades numéricas adequadas.

Método de Ralston

Estratégia de Implementação:

O método de Ralston foi implementado como uma variante dos métodos de Runge-Kutta de segunda ordem, buscando um equilíbrio entre precisão computacional e eficiência numérica. Diferente do método de Euler padrão que utiliza apenas uma estimativa da inclinação, o método de Ralston usa uma abordagem de duas etapas que combina duas estimativas diferentes da derivada para obter uma aproximação mais precisa.

A fundamentação matemática do método se baseia no cálculo de dois coeficientes principais:

- **k_1**: Corresponde à inclinação no ponto inicial do intervalo ($x_\square$, $y_\square$)
- **k_2**: Representa a inclinação em um ponto intermediário especificamente localizado em 75% do intervalo ($x_\square + \frac{3}{4}h$, $y_\square + \frac{3}{4}h \cdot k_1$)

A atualização do valor de y é realizada através de uma combinação ponderada dessas duas inclinações, seguindo a fórmula característica do método: $y_{\square+1} = y_\square + (h/3) \cdot (k_1 + 2 \cdot k_2)$. Esta ponderação específica, onde k_2 tem peso duplo em relação a k_1, foi cuidadosamente derivada para minimizar o erro de truncamento local, tornando o método de Ralston particularmente eficiente dentro da classe dos métodos de segunda ordem.

Na implementação prática, desenvolvi uma função dedicada ralston() que segue a mesma estrutura modular dos demais métodos numéricos já implementados. A função recebe como parâmetros a equação diferencial f(x,y), os valores iniciais x0 e y0, o tamanho do passo h e o número de iterações n. Durante a execução, mantenho o rastreamento completo da solução através de duas listas que armazenam sequencialmente todos os valores de x e y calculados.

Um aspecto importante da implementação foi garantir uma avaliação correta do ponto intermediário para o cálculo de k_2. Especificamente, o ponto ($x_\square + \frac{3}{4}h$, $y_\square + \frac{3}{4}h \cdot k_1$) deve ser calculado com precisão, o que exigiu atenção aos detalhes, especialmente no que diz respeito aos fatores fracionários envolvidos. A estrutura do código foi organizada para realçar claramente as três etapas principais: cálculo de k_1, cálculo de k_2 e atualização final de y.

Para manter a consistência com o restante do projeto, a implementação do método de Ralston reutiliza integralmente as funções auxiliares de leitura de entrada e gravação de saída (embora em outro código), garantindo

compatibilidade total com o formato estabelecido e permitindo mais facilmente que sejam feitas comparações diretas com outros métodos numéricos.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** segue um formato simples e direto, contendo em cada linha as informações necessárias para a execução do método:

1. Expressão da função diferencial f(x, y) em formato compatível com Python
2. Valor inicial de x (x_0)
3. Valor inicial de y (y_0)
4. Tamanho do passo de integração (h)
5. Número de iterações (n)

O arquivo de saída (**saida.txt**) é gerado automaticamente após a execução do método e contém:

1. Uma linha inicial indicando o método utilizado: "Método: Ralston"
2. Sequência de linhas com os pares (x, y) calculados em cada iteração, formatados com seis casas decimais

Dificuldades enfrentadas

A implementação do método de Ralston apresentou alguns desafios particulares que exigiram atenção cuidadosa aos detalhes matemáticos e de programação. Um dos principais pontos de atenção foi garantir a implementação correta da fórmula específica do método, que utiliza coeficientes fracionários (¾ e ⅓) em locais estratégicos do algoritmo. Esses valores não são arbitrários, mas resultam de uma otimização matemática para minimizar o erro, portanto qualquer imprecisão em sua implementação comprometeria a eficácia do método.

Outra dificuldade foi assegurar a distinção clara entre o método de Ralston e outras técnicas similares de segunda ordem, como, por exemplo, o método de Heun. Embora todos compartilhem uma mesma estrutura básica de duas

estimativas de inclinação, cada um possui coeficientes diferentes que refletem estratégias distintas de ponderação. Foi necessário estudar cuidadosamente a fundamentação teórica de cada método para evitar confusões e garantir que a implementação fosse fiel às especificações do método de Ralston.

A questão da precisão numérica também demandou atenção especial. Os cálculos envolvendo múltiplas operações com números de ponto flutuante e coeficientes fracionários podem potencialmente acumular erros de arredondamento. Para mitigar esse problema, optei por manter uma estrutura de cálculo sequencial e evitar operações desnecessárias que pudessem degradar a precisão.

Por fim, a integração do método na estrutura existente do projeto exigiu adaptações para garantir compatibilidade total com o sistema de entrada e saída, mantendo ao mesmo tempo as características únicas da implementação. A validação dos resultados através de comparações com soluções analíticas já conhecidas e com outros métodos numéricos foi essencial para verificar a corretude da implementação e assegurar que o método de Ralston estava realmente produzindo resultados com a precisão esperada para um método de segunda ordem.

Método de Runge-Kutta de 3° ordem

Estratégia de Implementação:

A implementação do método de Runge-Kutta de 3ª ordem foi desenvolvida seguindo a formulação matemática clássica deste método numérico, que busca melhorar significativamente a precisão das aproximações em relação aos métodos de ordem inferior. A principal vantagem dessa abordagem está na utilização de três estimativas de inclinação, que são cuidadosamente calculadas em pontos estratégicos do intervalo, permitindo uma aproximação mais refinada da solução da equação diferencial.

A lógica central do algoritmo baseia-se no cálculo de três coeficientes distintos:

- **k_1**: Representa a inclinação no ponto inicial do intervalo (xn, yn)
- **k_2**: Corresponde à inclinação no ponto médio do intervalo, utilizando k_1 para estimar o valor de y nessa posição
- **k_3**: Reflete a inclinação em um ponto avançado do intervalo, combinando informações de k_1 e k_2 de forma específica

A atualização do valor de y segue a fórmula característica do método:

$y_{n+1} = y_n + (h/6) * (k1 + 4k2 + k3)$. Essa combinação específica de pesos (1, 4, 1) foi derivada matematicamente para cancelar termos de erro até a terceira ordem, proporcionando uma maior precisão com um custo computacional moderado.

Na implementação prática, implementei o código de forma modular, criando uma função dedicada runge_kutta_3 que recebe os mesmos parâmetros básicos dos métodos anteriores, que são a função diferencial f(x,y), os valores iniciais x_0 e y_0, o tamanho do passo h e o número de iterações n. Durante a execução, mantenho o histórico completo da solução em duas listas separadas que armazenam todos os valores calculados de x e y.

Um aspecto importante da implementação foi garantir uma avaliação correta dos pontos intermediários para o cálculo de k_2 e k_3. Especificamente, o ponto para k_2 é $(x_n + h/2, y_n + (h * k1)/2)$, enquanto para k_3 é $(x_n + h, y_n - h * k1 + 2h * k2)$. Esta última expressão exigiu uma atenção especial devido à combinação não trivial dos coeficientes anteriores.

Para manter a consistência com o ecossistema de métodos já implementados, utilizei integralmente as funções auxiliares de leitura do arquivo de entrada e gravação do arquivo de saída (embora em códigos diferentes). Isso garante

total compatibilidade com o formato estabelecido e permite comparações diretas entre diferentes métodos numéricos. A estrutura do código foi organizada para destacar claramente as quatro etapas principais: cálculo de k_1, cálculo de k_2, cálculo de k_3 e atualização final de y.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** do método de Runge-Kutta de 3ª ordem utiliza exatamente o mesmo formato de arquivo de entrada estabelecido para os demais métodos, padronizado com as seguintes informações nas linhas:

1. Expressão matemática da função diferencial f(x, y) em sintaxe Python válida
2. Valor inicial da variável independente (x_0)
3. Valor inicial da variável dependente (y_0)
4. Tamanho do passo de integração (h)
5. Número total de iterações a serem executadas (n)

O arquivo de saída (**saida.txt**) gerado pelo método de Runge-Kutta de 3ª ordem segue o mesmo padrão estabelecido:

1. Linha inicial identificando o método utilizado: "Método: Runge-Kutta 3ª Ordem"
2. Sequência de linhas contendo os pares ordenados (x, y) calculados em cada passo, formatados com seis casas decimais de precisão

Dificuldades enfrentadas

A implementação do método de Runge-Kutta de 3ª ordem apresentou desafios específicos relacionados à complexidade aumentada em comparação com métodos de ordem inferior. A principal dificuldade estava na implementação correta da fórmula para o cálculo de k_3, que envolve uma combinação não intuitiva dos coeficientes anteriores: $y_n - h * k1 + 2h * k2$. Esta expressão exigiu uma maior atenção para garantir que os sinais e coeficientes estivessem exatamente como definido na formulação matemática do método.

Outro ponto que demandou certo cuidado foi a compreensão da lógica por trás da ponderação dos coeficientes na fórmula final. Os pesos (1, 4, 1) divididos por 6 não são arbitrários, mas resultam de um desenvolvimento matemático sofisticado para cancelar termos de erro. Entender esta fundamentação teórica foi importante para implementar corretamente o método.

A questão da precisão numérica se tornou mais crítica neste método devido ao maior número de operações envolvidas em cada iteração. Cada cálculo intermediário introduz possíveis erros de arredondamento, que podem ser acumulados ao longo das iterações. Para mitigar este problema, busquei uma estrutura de cálculo sequencial clara e evitei operações redundantes que pudessem degradar a precisão.

A validação dos resultados também foi mais complexa em comparação com alguns métodos mais simples. Além de verificar outras soluções analíticas conhecidas, foi necessário comparar com implementações de referência e com outros métodos numéricos para garantir que a precisão esperada de terceira ordem estava sendo alcançada. Esta verificação foi essencial para confirmar a corretude da implementação.

Por fim, a integração do método na estrutura existente do projeto exigiu adaptações para garantir que, apesar da maior complexidade interna, a interface externa permanecesse simples e consistente com os outros métodos. A manutenção da compatibilidade total com o sistema de entrada e saída foi um requisito importante que auxiliou em várias decisões de implementação.

Método de Runge-Kutta de 4° ordem

Estratégia de Implementação:

A implementação do método de Runge-Kutta de 4ª ordem foi desenvolvida seguindo a formulação clássica deste algoritmo, conhecida por seu equilíbrio entre precisão computacional e eficiência numérica. Considerado um dos métodos mais utilizados na prática para resolução de equações diferenciais ordinárias, essa abordagem emprega quatro estimativas de inclinação calculadas em pontos estrategicamente posicionados ao longo do intervalo, permitindo uma aproximação altamente precisa da solução.

A lógica computacional do algoritmo baseia-se no cálculo sequencial de quatro coeficientes distintos:

- **k_1**: Representa a inclinação no ponto inicial do intervalo (xn, yn)
- **k_2**: Corresponde à inclinação no ponto médio do intervalo, utilizando k_1 para estimar o valor de y nessa posição intermediária
- **k_3**: Reflete uma segunda estimativa no ponto médio, porém utilizando k_2 como referência para o cálculo da inclinação
- **k_4**: Representa a inclinação no ponto final do intervalo, baseando-se na estimativa fornecida por k_3

A atualização do valor de y segue a fórmula característica do método: $y_{n+1} = y_n + (h/6) * (k1 + 2k2 + 2k3 + k4)$. Esta combinação específica de pesos (1, 2, 2, 1) foi matematicamente otimizada para cancelar termos de erro até a quarta ordem, proporcionando uma ótima precisão com um custo computacional relativamente moderado, dada a qualidade dos resultados obtidos.

Na implementação prática, implementei o código de forma modular, criando uma função dedicada runge_kutta_4 que mantém a mesma interface dos métodos anteriores (embora em códigos diferentes), recebendo a função diferencial f(x,y), os valores iniciais x_0 e y_0, o tamanho do passo h e o número de iterações n. Durante a execução, o histórico completo da solução é mantido em duas estruturas de dados separadas que armazenam todos os valores calculados de x e y.

Um aspecto particularmente importante da implementação foi garantir a avaliação correta dos pontos para cada coeficiente, especialmente para k_2 e k_3 que compartilham a mesma coordenada x $(x_n + h/2)$ mas utilizam diferentes estimativas para y. Essa avaliação no ponto médio é uma característica marcante do método que contribui significativamente para sua precisão.

Para manter a consistência com os métodos já implementados, reutilizei integralmente as funções auxiliares de leitura do arquivo de entrada e gravação do arquivo de saída (embora em códigos diferentes). Esta decisão garante total compatibilidade com o formato estabelecido e facilita comparações diretas entre diferentes métodos numéricos. A estrutura do código foi organizada para destacar claramente as cinco etapas principais: cálculo de k_1, cálculo de k_2, cálculo de k_3, cálculo de k_4 e atualização final de y.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** do método de Runge-Kutta de 4ª ordem utiliza exatamente o mesmo formato de arquivo de entrada estabelecido para os demais métodos, padronizado com as seguintes informações nas linhas:

1. Expressão matemática da função diferencial f(x, y) em sintaxe Python válida
2. Valor inicial da variável independente (x_0)
3. Valor inicial da variável dependente (y_0)
4. Tamanho do passo de integração (h)
5. Número total de iterações a serem executadas (n)

O arquivo de saída (**saida.txt**) gerado pelo método de Runge-Kutta de 4ª ordem segue o mesmo padrão estabelecido:

1. Linha inicial identificando o método utilizado: "Método: Runge-Kutta 4ª Ordem"
2. Sequência de linhas contendo os pares ordenados (x, y) calculados em cada passo, formatados com seis casas decimais de precisão

Dificuldades enfrentadas

A implementação do método de Runge-Kutta de 4ª ordem apresentou desafios específicos relacionados à sua maior complexidade algorítmica em comparação com métodos de ordem inferior. A principal dificuldade estava na coordenação correta dos quatro cálculos de inclinação, cada um dependendo dos resultados dos cálculos anteriores de maneira específica. Particularmente,

foi necessário atenção para garantir que k_2 e k_3 fossem calculados corretamente, já que ambos se referem ao mesmo ponto x $(x_n + h/2)$ mas utilizam diferentes aproximações para y.

Outro ponto que demandou cuidado foi a implementação exata da fórmula de atualização final, com seus coeficientes específicos (1, 2, 2, 1) divididos por 6. A compreensão da lógica matemática por trás dessa ponderação foi essencial para a implementação do método.

A questão da precisão numérica foi especialmente relevante devido ao maior número de operações aritméticas envolvidas em cada iteração. Cada cálculo intermediário pode introduzir possíveis erros de arredondamento que, embora pequenos, poderiam se acumular ao longo de muitas iterações. Para mitigar esse risco, utilizei uma estratégia de cálculo sequencial bem estruturada e evitei operações redundantes ou numericamente instáveis.

A validação dos resultados exigiu uma abordagem abrangente, incluindo comparação com soluções analíticas conhecidas, verificação contra implementações de referência confiáveis e análise de convergência com diferentes tamanhos de passo. Essa validação foi crucial para confirmar que o método estava realmente atingindo a precisão de quarta ordem esperada.

Concluindo, a integração do método na estrutura existente do projeto representou um certo desafio. Era necessário equilibrar a complexidade interna do algoritmo com a simplicidade da interface externa, mantendo ao mesmo tempo a consistência com os outros métodos implementados. A decisão de preservar a mesma estrutura de entrada e saída, apesar da maior sofisticação interna, facilitou significativamente a usabilidade e comparação entre métodos.

Problemas Teste (Propostos pelo professor)

Problema 12.2:

12.2 - *Um corpo com uma massa inicial de 200kg é acelerado por uma força constante de 200N. A massa decresce a uma taxa de 1 kg/s. Se o corpo está em repouso em $t = 0$ encontre sua velocidade ao final de 50s. Sabe-se que a equação diferencial é dada por:*

$$\frac{dv}{dt} = \frac{2000}{200 - t} .$$

Resolva o problema usando:

a) *método de Euler,*

b) *método de Euler melhorado,*

c) *método de Nystrom.*

Observação: A solução exata desse **(p.v.i.)** é:

$$v = 2000log\left[\frac{200}{200 - t}\right] ,$$

de modo que $v(50) = 575.36$.

Entrada:

Como mantive o mesmo padrão para todos os arquivos de entrada, a entrada será a mesma para todos:

```
2000/(200 - x)
0
0
0.1
500
```

Saídas

OBS: Os prints mostram apenas os 11 primeiros e últimos resultados da saída. O restante dos valores podem ser vistos através da execução do código do respectivo método.

Saída por Euler:

```
Método: Euler
x = 0.000000, y = 0.000000
x = 0.100000, y = 1.000000
x = 0.200000, y = 2.000500
x = 0.300000, y = 3.001501
x = 0.400000, y = 4.003004
x = 0.500000, y = 5.005008
x = 0.600000, y = 6.007514
x = 0.700000, y = 7.010523
x = 0.800000, y = 8.014035
x = 0.900000, y = 9.018051
x = 1.000000, y = 10.022572
```

… … …

```
x = 49.000000, y = 561.912839
x = 49.100000, y = 563.237343
x = 49.200000, y = 564.562724
x = 49.300000, y = 565.888984
x = 49.400000, y = 567.216124
x = 49.500000, y = 568.544145
x = 49.600000, y = 569.873048
x = 49.700000, y = 571.202836
x = 49.800000, y = 572.533508
x = 49.900000, y = 573.865066
x = 50.000000, y = 575.197511
```

Saída por Euler Modificado:

```
Método: Euler Modificado
x = 0.000000, y = 0.000000
x = 0.100000, y = 1.000250
x = 0.200000, y = 2.001001
x = 0.300000, y = 3.002252
x = 0.400000, y = 4.004006
x = 0.500000, y = 5.006261
x = 0.600000, y = 6.009018
x = 0.700000, y = 7.012279
x = 0.800000, y = 8.016043
x = 0.900000, y = 9.020311
x = 1.000000, y = 10.025084
```

… … …

```
x = 49.000000, y = 562.075091
x = 49.100000, y = 563.400033
x = 49.200000, y = 564.725854
x = 49.300000, y = 566.052554
x = 49.400000, y = 567.380134
x = 49.500000, y = 568.708597
x = 49.600000, y = 570.037942
x = 49.700000, y = 571.368172
x = 49.800000, y = 572.699287
x = 49.900000, y = 574.031288
x = 50.000000, y = 575.364177
```

Saída por Heun:

```
Método: Heun
x = 0.000000, y = 0.000000
x = 0.100000, y = 1.000250
x = 0.200000, y = 2.001001
x = 0.300000, y = 3.002252
x = 0.400000, y = 4.004006
x = 0.500000, y = 5.006261
x = 0.600000, y = 6.009018
x = 0.700000, y = 7.012279
x = 0.800000, y = 8.016043
x = 0.900000, y = 9.020311
x = 1.000000, y = 10.025084
```

… … …

```
x = 49.000000, y = 562.075091
x = 49.100000, y = 563.400033
x = 49.200000, y = 564.725854
x = 49.300000, y = 566.052554
x = 49.400000, y = 567.380134
x = 49.500000, y = 568.708597
x = 49.600000, y = 570.037942
x = 49.700000, y = 571.368172
x = 49.800000, y = 572.699287
x = 49.900000, y = 574.031288
x = 50.000000, y = 575.364177
```

Saída por Ralston:

```
Método: Ralston
x = 0.000000, y = 0.000000
x = 0.100000, y = 1.000250
x = 0.200000, y = 2.001001
x = 0.300000, y = 3.002252
x = 0.400000, y = 4.004005
x = 0.500000, y = 5.006260
x = 0.600000, y = 6.009018
x = 0.700000, y = 7.012279
x = 0.800000, y = 8.016043
x = 0.900000, y = 9.020311
x = 1.000000, y = 10.025084
```

… … …

```
x = 49.000000, y = 562.075067
x = 49.100000, y = 563.400009
x = 49.200000, y = 564.725830
x = 49.300000, y = 566.052530
x = 49.400000, y = 567.380110
x = 49.500000, y = 568.708573
x = 49.600000, y = 570.037918
x = 49.700000, y = 571.368148
x = 49.800000, y = 572.699262
x = 49.900000, y = 574.031264
x = 50.000000, y = 575.364153
```

Saída por Runge-Kutta de 3° ordem:

```
Método: Runge-Kutta 3ª Ordem
x = 0.000000, y = 0.000000
x = 0.100000, y = 1.000250
x = 0.200000, y = 2.001001
x = 0.300000, y = 3.002252
x = 0.400000, y = 4.004005
x = 0.500000, y = 5.006260
x = 0.600000, y = 6.009018
x = 0.700000, y = 7.012279
x = 0.800000, y = 8.016043
x = 0.900000, y = 9.020311
x = 1.000000, y = 10.025084
```

… … …

```
x = 49.000000, y = 562.075059
x = 49.100000, y = 563.400002
x = 49.200000, y = 564.725822
x = 49.300000, y = 566.052522
x = 49.400000, y = 567.380102
x = 49.500000, y = 568.708565
x = 49.600000, y = 570.037910
x = 49.700000, y = 571.368140
x = 49.800000, y = 572.699254
x = 49.900000, y = 574.031256
x = 50.000000, y = 575.364145
```

Saída por Runge-Kutta de 4° ordem:

```
Método: Runge-Kutta 4ª Ordem
x = 0.000000, y = 0.000000
x = 0.100000, y = 1.000250
x = 0.200000, y = 2.001001
x = 0.300000, y = 3.002252
x = 0.400000, y = 4.004005
x = 0.500000, y = 5.006260
x = 0.600000, y = 6.009018
x = 0.700000, y = 7.012279
x = 0.800000, y = 8.016043
x = 0.900000, y = 9.020311
x = 1.000000, y = 10.025084
```

… … …

```
x = 49.000000, y = 562.075059
x = 49.100000, y = 563.400002
x = 49.200000, y = 564.725822
x = 49.300000, y = 566.052522
x = 49.400000, y = 567.380102
x = 49.500000, y = 568.708565
x = 49.600000, y = 570.037910
x = 49.700000, y = 571.368140
x = 49.800000, y = 572.699254
x = 49.900000, y = 574.031256
x = 50.000000, y = 575.364145
```

Problema 12.12:

12.12 - *A corrente i num circuito LR num instante t qualquer depois que uma chave é ligada em $t = 0$ pode ser expressa pela equação:*

$$\frac{di}{dt} = \frac{(E sen\ \omega t - R)}{L},$$

onde $E = 50$ Volts, $L = 1$ Henry, $\omega = 300$, $R = 50$ Ohms e a condição inicial é que $i(0) = 0$. Resolva numericamente o **(p.v.i.)** *por um método de Runge-Kutta de ordem 3 e compare sua solução com a solução analítica:*

$$i = \frac{E}{Z^2}(R sen\ \omega t - \omega L cos\ \omega t + \omega L e^{-Rt/L},$$

onde $Z = \sqrt{R^2 + \omega^2 L^2}$.

Entrada:

Como mantive o mesmo padrão para todos os arquivos de entrada, a entrada será a mesma para todos:

```
(50*math.sin(300*x) - 50*y)/1.0
0
0
0.001
100
```

Saídas

OBS: Os prints mostram apenas os 11 primeiros e últimos resultados da saída. O restante dos valores podem ser vistos através da execução do código do respectivo método.

Saída por Euler:

```
Método: Euler
x = 0.000000, y = 0.000000
x = 0.001000, y = 0.000000
x = 0.002000, y = 0.014776
x = 0.003000, y = 0.042269
x = 0.004000, y = 0.079322
x = 0.005000, y = 0.121958
x = 0.006000, y = 0.165735
x = 0.007000, y = 0.206141
x = 0.008000, y = 0.238994
x = 0.009000, y = 0.260817
x = 0.010000, y = 0.269146
```

… … …

```
x = 0.090000, y = 0.054005
x = 0.091000, y = 0.099124
x = 0.092000, y = 0.135534
x = 0.093000, y = 0.159976
x = 0.094000, y = 0.170260
x = 0.095000, y = 0.165460
x = 0.096000, y = 0.146000
x = 0.097000, y = 0.113610
x = 0.098000, y = 0.071180
x = 0.099000, y = 0.022493
x = 0.100000, y = -0.028106
```

Saída por Euler Modificado:

```
Método: Euler Modificado
x = 0.000000, y = 0.000000
x = 0.001000, y = 0.007388
x = 0.002000, y = 0.028163
x = 0.003000, y = 0.059783
x = 0.004000, y = 0.098774
x = 0.005000, y = 0.141032
x = 0.006000, y = 0.182193
x = 0.007000, y = 0.218020
x = 0.008000, y = 0.244780
x = 0.009000, y = 0.259573
x = 0.010000, y = 0.260597
```

… … …

```
x = 0.090000, y = 0.074804
x = 0.091000, y = 0.114555
x = 0.092000, y = 0.144229
x = 0.093000, y = 0.161168
x = 0.094000, y = 0.163852
x = 0.095000, y = 0.152034
x = 0.096000, y = 0.126763
x = 0.097000, y = 0.090291
x = 0.098000, y = 0.045869
x = 0.099000, y = -0.002540
x = 0.100000, y = -0.050617
```

Saída por Heun:

```
Método: Heun
x = 0.000000, y = 0.000000
x = 0.001000, y = 0.007388
x = 0.002000, y = 0.028163
x = 0.003000, y = 0.059783
x = 0.004000, y = 0.098774
x = 0.005000, y = 0.141032
x = 0.006000, y = 0.182193
x = 0.007000, y = 0.218020
x = 0.008000, y = 0.244780
x = 0.009000, y = 0.259573
x = 0.010000, y = 0.260597
```

… … …

```
x = 0.090000, y = 0.074804
x = 0.091000, y = 0.114555
x = 0.092000, y = 0.144229
x = 0.093000, y = 0.161168
x = 0.094000, y = 0.163852
x = 0.095000, y = 0.152034
x = 0.096000, y = 0.126763
x = 0.097000, y = 0.090291
x = 0.098000, y = 0.045869
x = 0.099000, y = -0.002540
x = 0.100000, y = -0.050617
```

Saída por Ralston:

```
Método: Ralston
x = 0.000000, y = 0.000000
x = 0.001000, y = 0.007437
x = 0.002000, y = 0.028337
x = 0.003000, y = 0.060146
x = 0.004000, y = 0.099366
x = 0.005000, y = 0.141870
x = 0.006000, y = 0.183270
x = 0.007000, y = 0.219303
x = 0.008000, y = 0.246213
x = 0.009000, y = 0.261088
x = 0.010000, y = 0.262112
```

… … …

```
x = 0.090000, y = 0.075258
x = 0.091000, y = 0.115236
x = 0.092000, y = 0.145078
x = 0.093000, y = 0.162110
x = 0.094000, y = 0.164804
x = 0.095000, y = 0.152911
x = 0.096000, y = 0.127488
x = 0.097000, y = 0.090800
x = 0.098000, y = 0.046117
x = 0.099000, y = -0.002575
x = 0.100000, y = -0.050931
```

Saída por Runge-Kutta de 3° ordem:

```
Método: Runge-Kutta 3ª Ordem
x = 0.000000, y = 0.000000
x = 0.001000, y = 0.007319
x = 0.002000, y = 0.028150
x = 0.003000, y = 0.059940
x = 0.004000, y = 0.099192
x = 0.005000, y = 0.141776
x = 0.006000, y = 0.183293
x = 0.007000, y = 0.219468
x = 0.008000, y = 0.246532
x = 0.009000, y = 0.261556
x = 0.010000, y = 0.262711
```

… … …

```
x = 0.090000, y = 0.075019
x = 0.091000, y = 0.115160
x = 0.092000, y = 0.145172
x = 0.093000, y = 0.162365
x = 0.094000, y = 0.165198
x = 0.095000, y = 0.153409
x = 0.096000, y = 0.128045
x = 0.097000, y = 0.091367
x = 0.098000, y = 0.046643
x = 0.099000, y = -0.002137
x = 0.100000, y = -0.050620
```

Saída por Runge-Kutta de 4° ordem:

```
Método: Runge-Kutta 4ª Ordem
x = 0.000000, y = 0.000000
x = 0.001000, y = 0.007321
x = 0.002000, y = 0.028153
x = 0.003000, y = 0.059944
x = 0.004000, y = 0.099197
x = 0.005000, y = 0.141781
x = 0.006000, y = 0.183297
x = 0.007000, y = 0.219471
x = 0.008000, y = 0.246534
x = 0.009000, y = 0.261557
x = 0.010000, y = 0.262710
```

… … …

```
x = 0.090000, y = 0.075023
x = 0.091000, y = 0.115164
x = 0.092000, y = 0.145174
x = 0.093000, y = 0.162366
x = 0.094000, y = 0.165197
x = 0.095000, y = 0.153407
x = 0.096000, y = 0.128042
x = 0.097000, y = 0.091362
x = 0.098000, y = 0.046638
x = 0.099000, y = -0.002142
x = 0.100000, y = -0.050625
```

Problema 12.16:

12.16 - *Engenheiros ambientais e biomédicos precisam frequentemente prever o resultado de uma relação predador-presa ou hospedeiro-parasita. Um modelo simples para esse tipo de relação é dado pelas equações de Lotka-Volterra:*

$$\frac{dH}{dt} = g_1 H - d_1 P H \ ,$$

$$\frac{dP}{dt} = -d_2 P + g_2 P H \ .$$

onde H e P são, respectivamente, por exemplo, o número de hospedeiros e parasitas presentes. As constantes d e g representam as taxas de mortalidade e crescimento, respectivamente. O índice 1 refere-se ao hospedeiro e o índice 2 ao parasita. Observe que essas equações formam um sistema de equações acopladas. Sabendo que no instante $t_0 = 0$, os valores de P e H são , respectivamente, 5 e 20, e que $g_1 = 1, d_1 = 0.1, g_2 = 0.02$ e $d_2 = 0.5$, utilize um método numérico para calcular os valores de H e P de 0 até $2s$, usando passo de $0.01s$.

Entrada:

Como mantive o mesmo padrão para todos os arquivos de entrada, a entrada será a mesma para todos:

```
-0.5*x + 0.02*x*y
0
20
0.01
200
```

Saídas

OBS: Os prints mostram apenas os 11 primeiros e últimos resultados da saída. O restante dos valores podem ser vistos através da execução do código do respectivo método.

Saída por Euler:

```
Método: Euler
x = 0.000000, y = 20.000000
x = 0.010000, y = 20.000000
x = 0.020000, y = 19.999990
x = 0.030000, y = 19.999970
x = 0.040000, y = 19.999940
x = 0.050000, y = 19.999900
x = 0.060000, y = 19.999850
x = 0.070000, y = 19.999790
x = 0.080000, y = 19.999720
x = 0.090000, y = 19.999640
x = 0.100000, y = 19.999550
```

… … …

```
x = 1.900000, y = 19.817211
x = 1.910000, y = 19.815241
x = 1.920000, y = 19.813261
x = 1.930000, y = 19.811269
x = 1.940000, y = 19.809266
x = 1.950000, y = 19.807252
x = 1.960000, y = 19.805227
x = 1.970000, y = 19.803191
x = 1.980000, y = 19.801143
x = 1.990000, y = 19.799084
x = 2.000000, y = 19.797014
```

Saída por Euler Modificado:

```
Método: Euler Modificado
x = 0.000000, y = 20.000000
x = 0.010000, y = 19.999995
x = 0.020000, y = 19.999980
x = 0.030000, y = 19.999955
x = 0.040000, y = 19.999920
x = 0.050000, y = 19.999875
x = 0.060000, y = 19.999820
x = 0.070000, y = 19.999755
x = 0.080000, y = 19.999680
x = 0.090000, y = 19.999595
x = 0.100000, y = 19.999500
```

… … …

```
x = 1.900000, y = 19.816202
x = 1.910000, y = 19.814227
x = 1.920000, y = 19.812240
x = 1.930000, y = 19.810243
x = 1.940000, y = 19.808234
x = 1.950000, y = 19.806214
x = 1.960000, y = 19.804183
x = 1.970000, y = 19.802140
x = 1.980000, y = 19.800087
x = 1.990000, y = 19.798022
x = 2.000000, y = 19.795946
```

Saída por Heun:

```
Método: Heun
x = 0.000000, y = 20.000000
x = 0.010000, y = 19.999995
x = 0.020000, y = 19.999980
x = 0.030000, y = 19.999955
x = 0.040000, y = 19.999920
x = 0.050000, y = 19.999875
x = 0.060000, y = 19.999820
x = 0.070000, y = 19.999755
x = 0.080000, y = 19.999680
x = 0.090000, y = 19.999595
x = 0.100000, y = 19.999500
```

… … …

```
x = 1.900000, y = 19.816202
x = 1.910000, y = 19.814227
x = 1.920000, y = 19.812240
x = 1.930000, y = 19.810243
x = 1.940000, y = 19.808234
x = 1.950000, y = 19.806214
x = 1.960000, y = 19.804183
x = 1.970000, y = 19.802140
x = 1.980000, y = 19.800087
x = 1.990000, y = 19.798022
x = 2.000000, y = 19.795946
```

Saída por Ralston:

```
Método: Ralston
x = 0.000000, y = 20.000000
x = 0.010000, y = 19.999995
x = 0.020000, y = 19.999980
x = 0.030000, y = 19.999955
x = 0.040000, y = 19.999920
x = 0.050000, y = 19.999875
x = 0.060000, y = 19.999820
x = 0.070000, y = 19.999755
x = 0.080000, y = 19.999680
x = 0.090000, y = 19.999595
x = 0.100000, y = 19.999500
```

… … …

```
x = 1.900000, y = 19.816202
x = 1.910000, y = 19.814227
x = 1.920000, y = 19.812241
x = 1.930000, y = 19.810243
x = 1.940000, y = 19.808234
x = 1.950000, y = 19.806214
x = 1.960000, y = 19.804183
x = 1.970000, y = 19.802141
x = 1.980000, y = 19.800087
x = 1.990000, y = 19.798022
x = 2.000000, y = 19.795946
```

Saída por Runge-Kutta de 3° ordem:

```
Método: Runge-Kutta 3ª Ordem
x = 0.000000, y = 20.000000
x = 0.010000, y = 19.999995
x = 0.020000, y = 19.999980
x = 0.030000, y = 19.999955
x = 0.040000, y = 19.999920
x = 0.050000, y = 19.999875
x = 0.060000, y = 19.999820
x = 0.070000, y = 19.999755
x = 0.080000, y = 19.999680
x = 0.090000, y = 19.999595
x = 0.100000, y = 19.999500
```

… … …

```
x = 1.900000, y = 19.816202
x = 1.910000, y = 19.814227
x = 1.920000, y = 19.812240
x = 1.930000, y = 19.810243
x = 1.940000, y = 19.808234
x = 1.950000, y = 19.806214
x = 1.960000, y = 19.804183
x = 1.970000, y = 19.802140
x = 1.980000, y = 19.800087
x = 1.990000, y = 19.798022
x = 2.000000, y = 19.795946
```

Saída por Runge-Kutta de 4° ordem:

```
Método: Runge-Kutta 4ª Ordem
x = 0.000000, y = 20.000000
x = 0.010000, y = 19.999995
x = 0.020000, y = 19.999980
x = 0.030000, y = 19.999955
x = 0.040000, y = 19.999920
x = 0.050000, y = 19.999875
x = 0.060000, y = 19.999820
x = 0.070000, y = 19.999755
x = 0.080000, y = 19.999680
x = 0.090000, y = 19.999595
x = 0.100000, y = 19.999500
```

… … …

```
x = 1.900000, y = 19.816202
x = 1.910000, y = 19.814227
x = 1.920000, y = 19.812240
x = 1.930000, y = 19.810243
x = 1.940000, y = 19.808234
x = 1.950000, y = 19.806214
x = 1.960000, y = 19.804183
x = 1.970000, y = 19.802140
x = 1.980000, y = 19.800087
x = 1.990000, y = 19.798022
x = 2.000000, y = 19.795946
```

Método Shooting

Estratégia de Implementação:

O método de Shooting foi implementado como uma abordagem para resolver problemas de valor de contorno em equações diferenciais ordinárias de segunda ordem. Essa técnica transforma um problema de valor de contorno em uma sequência de problemas de valor inicial, que são resolvidos iterativamente até que as condições de contorno sejam satisfeitas.

A implementação segue uma abordagem sistemática que combina dois elementos principais: o método de Runge-Kutta de 4ª ordem para resolver os problemas de valor inicial e o método da secante para ajustar iterativamente o "chute" inicial da derivada. A ideia é encontrar o valor correto da derivada inicial y'(a) que, quando utilizado na integração numérica, leva à condição de contorno desejada y(b).

O algoritmo começa com dois chutes iniciais para a derivada (s0 e s1). Para cada chute, o sistema de equações diferenciais de primeira ordem é resolvido usando o método de Runge-Kutta de 4ª ordem adaptado para sistemas. Os valores resultantes de y(b) são comparados com o valor desejado, e o erro é calculado como a diferença entre esses valores.

Utilizando o método da secante, um novo chute para a derivada inicial é calculado, com base nos erros dos dois chutes anteriores. Esse processo iterativo continua até que o erro seja menor que uma tolerância definida anteriormente ou até que um número máximo de iterações seja alcançado.

A implementação incluiu uma função especializada runge_kutta_4_ordem_edo2 que resolve sistemas de duas equações diferenciais de primeira ordem, calculando simultaneamente os valores de y e y' em cada passo. Essa função é essencial para o método, pois permite a integração numérica precisa do sistema transformado.

Para garantir robustez, o código inclui verificações de convergência e limites no número de iterações, prevenindo loops infinitos em casos onde a convergência não é alcançada. A estrutura modular do código facilita a manutenção e possíveis modificações, separando a lógica de integração numérica da lógica de ajuste do chute inicial.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** do método de Shooting requer um formato específico de entrada devido à natureza do problema de valor de contorno. O arquivo deve conter as seguintes informações, uma por linha:

1. Expressão da função que define a segunda derivada y'' = f(x, y, y') em sintaxe Python válida
2. Extremo inferior do intervalo (a)
3. Extremo superior do intervalo (b)
4. Condição de contorno no ponto inicial y(a)
5. Condição de contorno no ponto final y(b)
6. Primeiro chute para a derivada inicial y'(a)
7. Segundo chute para a derivada inicial y'(a)
8. Tamanho do passo de integração (h)

O arquivo de saída (**saida.txt**) gerado pelo método de Shooting segue o padrão estabelecido para os outros métodos:

1. Linha inicial identificando o método utilizado: "Método: Shooting"
2. Sequência de linhas contendo os pares ordenados (x, y) calculados em cada passo do intervalo, formatados com seis casas decimais de precisão

Dificuldades enfrentadas

A implementação do método de Shooting apresentou desafios significativos relacionados à sua natureza iterativa e à transformação matemática necessária. Uma das principais dificuldades foi a conversão correta da equação diferencial de segunda ordem em um sistema de duas equações de primeira ordem. Essa transformação exigiu uma compreensão clara das relações entre as variáveis e suas derivadas, e qualquer erro nesta etapa poderia comprometer toda a solução.

Outro ponto foi a implementação do método de Runge-Kutta de 4ª ordem adaptado para sistemas de equações. Enquanto a versão padrão lida com uma única equação diferencial, a versão para sistemas requer o cálculo simultâneo de múltiplas variáveis e suas derivadas, aumentando a complexidade do código e a possibilidade de erros.

A convergência do método da secante também exigiu uma atenção especial. Em alguns casos, os chutes iniciais fornecidos pelo usuário podem não levar a uma convergência rápida ou podem até causar divergência. Implementar

verificações e limites no número de iterações foi necessário para garantir que o programa não entrasse em loop infinito em alguns casos.

A precisão numérica foi uma preocupação ainda mais crítica neste método devido à sua natureza iterativa. Erros de arredondamento podem se acumular não apenas durante a integração numérica, mas também através das múltiplas iterações do método da secante. Encontrar um equilíbrio entre precisão e desempenho computacional foi necessário durante o desenvolvimento.

A validação dos resultados foi particularmente complexa para este método, já que problemas de valor de contorno frequentemente não possuem soluções analíticas simples para comparação. Foi necessário confiar em problemas testes e em verificações com outras implementações de referência para garantir a corretude da solução.

Concluindo, a integração do método na estrutura existente do projeto exigiu adaptações no formato de entrada para acomodar os parâmetros adicionais necessários (dois chutes iniciais para a derivada). Manter a consistência com a interface dos outros métodos enquanto adicionava uma certa complexidade adicional foi de grande valor.

Método de Diferenças Finitas

Estratégia de Implementação:

O método das diferenças finitas foi implementado como uma abordagem numérica para resolver problemas de valor de contorno em equações diferenciais ordinárias de segunda ordem. Essa técnica transforma uma equação diferencial contínua em um sistema discreto de equações algébricas, que pode ser resolvido utilizando métodos de álgebra linear.

A implementação segue uma abordagem baseada na discretização do intervalo de solução. O processo inicia dividindo o intervalo [a, b] em n subintervalos iguais, gerando n+1 pontos de discretização. Para cada ponto interno do domínio, a derivada segunda da função é aproximada utilizando a fórmula clássica de diferenças centrais de segunda ordem: $y''(x_i) \approx (y_{i+1} - 2y_i + y_{i-1})/h^2$.

Essa aproximação transforma a equação diferencial original em um sistema linear tridiagonal de n-1 equações para os n-1 valores desconhecidos de y nos pontos internos. As condições de contorno y(a) = ya e y(b) = yb são integradas diretamente no sistema, fixando os valores nas extremidades.

A implementação constrói uma matriz A de coeficientes que possui uma estrutura tridiagonal regular, com -2 na diagonal principal, 1 nas diagonais superior e inferior, e zeros em todas as outras posições. O vetor B do lado direito do sistema é calculado como $h^2 * f(x_i, y_i)$, onde f(x, y) representa o termo fonte da equação diferencial.

Para resolver o sistema linear resultante, a implementação utiliza a função np.linalg.solve da biblioteca NumPy, que implementa algoritmos numéricos para encontrar a solução exata do sistema. Essa abordagem direta evita a necessidade de métodos iterativos, proporcionando uma solução precisa com um esforço computacional menor.

Uma característica importante da implementação é a geração automática da malha de pontos x_i igualmente espaçados, o que simplifica a construção do sistema linear e garante a consistência da discretização. A solução obtida nos pontos internos é, então, combinada com as condições de contorno para formar o vetor completo de soluções.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada.txt)** do método das diferenças finitas requer um formato específico de entrada que define completamente o problema de valor de contorno:

1. Expressão da função f(x, y) que define o termo fonte da equação diferencial y'' = f(x, y)
2. Extremo inferior do intervalo (a)
3. Extremo superior do intervalo (b)
4. Condição de contorno no ponto inicial y(a)
5. Condição de contorno no ponto final y(b)
6. Número de subintervalos para a discretização (n)

O arquivo de saída (**saida.txt**) gerado pelo método das diferenças finitas segue o padrão estabelecido para os outros métodos:

1. Linha inicial identificando o método utilizado: "Método: Diferenças Finitas"
2. Sequência de linhas contendo os pares ordenados (x, y) calculados em cada ponto da discretização, formatados com seis casas decimais de precisão

Dificuldades enfrentadas

A implementação do método das diferenças finitas apresentou desafios específicos relacionados à construção e solução do sistema linear resultante da discretização. Uma das principais dificuldades foi garantir a correta construção da matriz de coeficientes, especialmente no tratamento das condições de contorno. A incorporação das condições y(a) e y(b) no sistema exigiu ajustes no vetor do lado direito, e qualquer erro nesta etapa poderia comprometer toda a solução.

Outro ponto foi a necessidade de fornecer uma estimativa inicial para os valores de y_i no cálculo do vetor B. Embora o método não seja iterativo nesse aspecto, a qualidade da solução pode ser afetada se a função f(x, y) for não-linear, exigindo potencialmente iterações adicionais para refinar a solução. A implementação utiliza uma aproximação linear simples, que funciona bem para problemas lineares mas pode requerer adaptações para casos não-lineares.

A escolha do número adequado de subintervalos também exigiu certa atenção. Um número muito pequeno de subintervalos pode levar a soluções imprecisas devido ao erro de truncamento, enquanto um número muito grande pode resultar em um sistema linear com matriz mal condicionada ou em custos computacionais desnecessários. Encontrar um equilíbrio entre precisão e desempenho foi uma consideração importante durante o desenvolvimento.

A solução do sistema linear com np.linalg.solve, embora eficiente, levantou questões sobre estabilidade numérica para matrizes mal condicionadas que podem surgir em certas discretizações. Implementar verificações de condicionamento e estratégias alternativas de solução para esses casos pode ser um aprimoramento valioso.

A validação dos resultados foi facilitada pela natureza algébrica do método, permitindo comparações diretas com soluções analíticas conhecidas para problemas lineares simples. No entanto, para problemas mais complexos, foi necessário confiar em verificações de consistência e em comparações com outros métodos numéricos para garantir a corretude da implementação.

Por fim, a integração do método na estrutura existente do projeto exigiu adaptações no formato de entrada para acomodar o parâmetro adicional (número de subintervalos).

Problemas Teste (Propostos pelo professor)

Os seguintes problemas tiveram seus valores extraídos do material do professor, disponível nos slides sobre Problema de Valores de contorno, Método shooting e Método das diferenças finitas. Porém, foi proposto que os problemas fossem resolvidos com 18 pontos internos.

Problema 1 (Método “Shooting”)

Para obter 18 pontos internos no método de Shooting, precisamos dividir o intervalo total (de 0 a 10) em 19 subintervalos. Porém, no arquivo de entrada, não informamos o número de pontos, mas sim o tamanho do passo (linha 8). Portanto, calculando o tamanho do passo, temos 10/19 ≈0.5263157894736842, esse valor será nosso tamanho de passo.

Entrada:

```
1 0.01*(20 - y)
2 0
3 10
4 40
5 200
6 10
7 20
8 0.5263157894736842
```

Saída:

```
Método: Shooting
x = 0.000000, y = 40.000000
x = 0.526316, y = 50.550013
x = 1.052632, y = 61.015420
x = 1.578947, y = 71.367237
x = 2.105263, y = 81.576795
x = 2.631579, y = 91.615819
x = 3.157895, y = 101.456508
x = 3.684211, y = 111.071607
x = 4.210526, y = 120.434489
x = 4.736842, y = 129.519224
x = 5.263158, y = 138.300650
x = 5.789474, y = 146.754450
x = 6.315789, y = 154.857211
x = 6.842105, y = 162.586492
x = 7.368421, y = 169.920888
x = 7.894737, y = 176.840087
x = 8.421053, y = 183.324926
x = 8.947368, y = 189.357446
x = 9.473684, y = 194.920940
x = 10.000000, y = 200.000000
```

Problema 2 (Método de Diferenças Finitas)

No Método das Diferenças Finitas, o número total de pontos na malha é n+1, onde n é o número de divisões (subintervalos). Portanto, teremos 18 + 1 = 19, que será o valor do nosso número de subintervalos.

Entrada:

```
0.01*(20 - y)
0
10
40
200
19
```

Saída:

```
Metodo: Diferencas Finitas
x = 0.000000, y = 40.000000
x = 0.526316, y = 50.319289
x = 1.052632, y = 60.559848
x = 1.578947, y = 70.698353
x = 2.105263, y = 80.711474
x = 2.631579, y = 90.575886
x = 3.157895, y = 100.268261
x = 3.684211, y = 109.765272
x = 4.210526, y = 119.043592
x = 4.736842, y = 128.079895
x = 5.263158, y = 136.850853
x = 5.789474, y = 145.333139
x = 6.315789, y = 153.503426
x = 6.842105, y = 161.338388
x = 7.368421, y = 168.814696
x = 7.894737, y = 175.909025
x = 8.421053, y = 182.598046
x = 8.947368, y = 188.858434
x = 9.473684, y = 194.666861
x = 10.000000, y = 200.000000
```

Considerações Finais

A realização deste trabalho proporcionou uma experiência prática valiosa na implementação e comparação de diferentes métodos numéricos para resolução de equações diferenciais. Mais do que aplicar fórmulas matemáticas, foi possível também compreender como decisões de implementação, como o tamanho do passo, a forma de discretização e o tratamento de condições de contorno, podem impactar diretamente a precisão e a eficiência das soluções obtidas.

Os métodos de passo simples, como Euler, Euler Modificado, Heun e Ralston, mostraram-se acessíveis e de implementação direta, sendo mais úteis para uma primeira aproximação do problema. No entanto, ficou evidente que métodos de ordem superior, como os de Runge-Kutta de 3ª e 4ª ordem, oferecem um ganho significativo em precisão com um custo computacional ainda razoável, sendo possivelmente melhores em aplicações que exigem maior confiabilidade dos resultados.

Para problemas com condições de contorno, duas abordagens distintas foram implementadas: o método do Shooting, que converte o problema em uma sequência de problemas de valor inicial, e o método das Diferenças Finitas, que discretiza a equação em um sistema linear. Enquanto o Shooting requer um processo iterativo de ajuste e depende de um bom chute inicial, o método das Diferenças Finitas oferece uma solução direta via álgebra linear, sendo muitas vezes mais estável e previsível.

Além dos resultados numéricos em si, este trabalho reforçou a importância de um código modular e com entradas e saídas padronizadas,facilitando a comparação entre métodos, a reprodução de testes e a manutenção do projeto. A integração entre conceitos matemáticos e prática de programação foi de grande valor, mostrando como a computação permite transformar teorias abstratas em ferramentas concretas para simulação e análise.

Concluindo, fica claro que não existe um método universalmente ideal ou perfeito, a escolha deve considerar a natureza do problema, as condições disponíveis e o equilíbrio desejado entre precisão, esforço computacional e complexidade de implementação. Esse trabalho não apenas solidificou o entendimento dos métodos estudados, mas também preparou terreno para a solução de problemas mais complexos em áreas como simulação científica, modelagem de sistemas dinâmicos e análise de dados.